

RTI website

Developer handbook

recycletechnologies.com: getting the code, setting up the database, running it locally, and sending changes to the dev server and the live site.

Repository	github.com/rti-website/rti-website (private), branch main
Live site	https://www.recycletechnologies.com (admin at /admin/)
Dev site	https://dev.recycletechnologies.com (admin at /admin/)
Server	dev1@192.168.90.152 (office network; one VM runs dev and live)
Written	28 September 2026

No passwords, keys or tokens are written in this document on purpose. Ask Asim for access; see section 2.

Contents

1. What this project is	3
2. Access you need	3
3. Install the tools	4
4. Get the code from GitHub	4
5. Set up your local database	5
6. Run it locally	6
7. Where things are	7
8. Rules that protect the site	7
9. Common tasks	8
10. Send changes to the dev server	9
11. Send changes to the live site	9
12. Facts about the server	10
13. When something goes wrong	11
14. Checklist for your first day	11

1. What this project is

The site is a **Next.js 16** app (App Router, Turbopack, Tailwind 4) that replaced the old WordPress site on 24 Sep 2026. About 600 URLs already rank on Google, so the most important rule is: **do not change what a page says or where it lives** unless that change was asked for (see section 8).

- **Public pages** are prerendered as static HTML at build time. Blog posts, category pages and partner location pages are read from PostgreSQL during the build.
- **The admin** ("Publisher") is at `/admin/`. It is one static page that talks to `/api/admin/*` route handlers. Posts, SEO fields, page copy (Admin → Pages), enquiries, the lead workflow, users, tracking settings and location pages are all edited there.
- **Data** lives in PostgreSQL. Uploaded images live in a folder on disk (`MEDIA_DIR`), not in git.
- **Email** (enquiry notifications, the residential auto reply, the agent assignment email) goes through SMTP (Mailtrap on the servers).

The three environments

	Local (your computer)	Dev server	Live site
Address	<code>http://localhost:3000</code>	<code>dev.recycletechnologies.com</code>	<code>www.recycletechnologies.com</code>
Folder	wherever you clone it	<code>~/apps/rti-website-dev</code>	<code>~/apps/rti-website</code>
Process	<code>npm run dev</code>	<code>pm2 rti-website-dev</code> , port 8106	<code>pm2 rti-website</code> , port 8201
Database	your own local copy	<code>rti_admin_dev</code>	<code>rti_live</code>
Images	<code>var/uploads/</code>	<code>var/uploads/</code> in the dev folder	<code>~/apps/rti-website-data/uploads</code>
Google	not reachable	blocked (noindex)	indexed

The normal flow of a change: **local → push to GitHub → dev server → check → live site**.

2. Access you need

What	Why	Who gives it
GitHub access to rti-website/rti-website	To clone, pull and push the code	Asim (already done)
An SSH login on 192.168.90.152	To deploy to dev and live, and to copy the database and images	Asim / the server admins. The server is on the office network: you need to be on it or on its VPN.
An admin account on dev and on live	To use <code>/admin/</code> on those sites	An administrator creates it in Admin → People & access
Figma file access (optional)	Only if you implement new designs	Asim

Important. Never put a password, API key or token in git, in this document, in a chat or in an email. The Mailtrap token and the Google Maps key exist only in the servers' settings files and in the admin. If you ever see one in the repository, tell Asim so it can be changed.

3. Install the tools

Tool	Version	Check with
Node.js	22.22 or newer (package.json requires it)	<code>node -v</code>
Git	any recent	<code>git --version</code>
PostgreSQL	18 (16 or newer is fine locally)	<code>psql --version</code>
A code editor	VS Code is what the team uses	

On **Windows** (PowerShell):

```
winget install -e --id OpenJS.NodeJS.LTS
winget install -e --id Git.Git
winget install -e --id PostgreSQL.PostgreSQL.18
```

The PostgreSQL installer asks for a password for the **postgres** user. Write it down; section 5 needs it. The installer usually does not put `psql` on PATH; its full path is `C:\Program Files\PostgreSQL\18\bin\psql.exe`.

On **macOS**:

```
brew install node git postgresql@18
brew services start postgresql@18
```

4. Get the code from GitHub

Windows PowerShell (the same commands work on macOS/Linux with your own path)

```
cd C:\Users\YOU\Desktop          # or wherever you keep projects
git clone https://github.com/rti-website/rti-website.git
cd rti-website
npm ci
```

- The first `git clone` asks you to sign in to GitHub. On Windows, Git Credential Manager opens a browser window; on macOS use `gh auth login` (GitHub CLI) or a personal access token. Never paste a token into a URL or a file.
- `npm ci` installs exactly the versions in `package-lock.json`. Use `npm ci`, not `npm install`, unless you are deliberately adding a package.
- The images the site uses (`public/images/`) are in git. Uploaded media (`var/uploads/`) is not; section 5 option B copies it.

Before you start work each day:

```
git checkout main
git pull origin main
npm ci                # only needed when package-lock.json changed
```

5. Set up your local database

5.1 Create .env.local

In the project folder, create a file named `.env.local` (it is git-ignored and never leaves your computer). `.env.example` explains every setting. For local work you need two lines:

```
.env.local
```

```
DATABASE_URL=postgres://postgres:YOUR_POSTGRES_PASSWORD@localhost:5432/rti_admin  
ADMIN_SESSION_SECRET=PASTE_THE_OUTPUT_OF_THE_COMMAND_BELOW
```

Generates the session secret

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Leave the SMTP lines out locally: enquiries are still saved and shown in the admin, only the email is skipped. Never copy a server's `.env.local` to your computer.

5.2 Option A: an empty database (quick start)

```
npm run db:setup -- --email you@example.com --name "Your Name" --password "a good password"
```

This finds PostgreSQL, creates the `rti_admin` database, applies every migration in `db/` (001 to 012 today), loads the categories and redirects, and creates your administrator account. It is safe to run again; it only applies what is missing. Optional sample content: `npm run db:seed` (remove it with `npm run db:seed -- --clear`).

The downside: there are no blog posts, so the blog, category pages and post URLs are empty. For real work, use option B.

5.3 Option B: a copy of the dev database and images (recommended)

This gives you the 300 plus imported blog posts, SEO fields, page copy and location pages exactly as on dev. It needs an SSH login on the server (or ask Asim to run step 1 and send you the two files).

Step 1, on the server (`ssh dev1@192.168.90.152`):

```
cd ~/apps/rti-website-dev  
DB=$(grep -E '^DATABASE_URL=' .env.local | cut -d= -f2- | tr -d ' ')  
pg_dump "$DB" --no-owner --no-privileges -f ~/rti-dev.sql  
M=$(sed -nE 's#^MEDIA_DIR=(.*)#\1#p' .env.local); M=${M:-var/uploads}  
tar czf ~/rti-uploads.tgz -C "$M" .  
ls -lh ~/rti-dev.sql ~/rti-uploads.tgz
```

Step 2, on your computer, in the project folder:

Windows PowerShell (on macOS/Linux use `var/uploads`)

```
scp dev1@192.168.90.152:~/rti-dev.sql .  
scp dev1@192.168.90.152:~/rti-uploads.tgz .  
mkdir var\uploads  
tar xzf rti-uploads.tgz -C var\uploads
```

Step 3, load the database (enter the postgres password when asked):

Windows PowerShell (on macOS: createdb rti_admin, then psql -d rti_admin ...)

```
$PG = "C:\Program Files\PostgreSQL\18\bin"
& "$PG\createdb.exe" -U postgres rti_admin
& "$PG\psql.exe" -U postgres -d rti_admin -v ON_ERROR_STOP=1 -q -f rti-dev.sql
& "$PG\psql.exe" -U postgres -d rti_admin -c "TRUNCATE sessions, leads, subscribers CASCADE;"
```

The last line removes logins, enquiries and newsletter sign ups from **your** copy: they are personal data you do not need. Then give yourself an administrator login and tidy up:

```
npm run user:add -- --login you@example.com --password "a good password" --name "Your Name"
del rti-dev.sql, rti-uploads.tgz
```

Step 4, delete the copies on the server (they contain password hashes):

on the server

```
rm ~/rti-dev.sql ~/rti-uploads.tgz
```

Important. On a database restored from a dump, **do not run** `npm run db:setup`. Its migration list starts empty on such a copy, so it tries to re-run old migrations and stops at 004. Apply new migrations with `psql -f db/NNN_name.sql` instead (section 9.2).

6. Run it locally

```
npm run dev
```

Open **http://localhost:3000**. The admin is **http://localhost:3000/admin/**; sign in with the account from section 5. Pages rebuild as you save files.

To check a production build locally (what the servers run):

```
npm run build
npm run verify:static
npm start # serves the build on http://localhost:3000
```

Checks to run before every push

Command	What it proves
<code>npm run typecheck</code>	TypeScript has no errors
<code>npm run lint</code>	No lint errors (warnings that were already there are fine)
<code>npm run build</code>	The site builds. Pages must show ○ (Static) or ● (SSG). Only <code>/api/*</code> and <code>/uploads</code> may show <code>f</code> .
<code>npm run verify:static</code>	"Build guard passed": every declared URL was prerendered, and the noindex guard is right
<code>npm run verify:assets</code>	Every image the code refers to exists

7. Where things are

Path	What is in it
CLAUDE.md	The project rules. Read it before your first change.
src/app/	Routes. One folder per explicit page; [...slug] builds MDX pages and blog posts.
src/app/api/admin/	Admin endpoints. Every one goes through guard() in src/lib/admin-route.ts.
src/components/sections/	Page sections (server components).
src/components/client/	The only place 'use client' is allowed. admin/ holds every admin screen.
src/data/	Page copy as the developers wrote it. Admin → Pages edits are stored as patches over it.
src/content/registry.ts	The list of documents Admin → Pages can edit.
src/lib/	urls, seo, db, auth, mail, page content, lead email, locations.
content/	MDX pages (legal pages, thank you, one post).
data/url-map.csv	Source of truth for URLs, titles, descriptions, H1s and 301s.
data/redirects.json	Generated from url-map.csv. Never edit by hand.
data/figma-assets.json	Every Figma image: where it goes and which node it came from.
db/	Numbered SQL migrations, 001 to 012 so far.
scripts/	Build and QA tools (plain node).
deploy/	Server notes and the production settings template.
var/	Uploads and the GeoIP file. Git-ignored.

8. Rules that protect the site

These come from CLAUDE.md. Each one exists because breaking it loses Google rankings or leaks something.

- **Never change a title, meta description or H1** that came from data/url-map.csv. Flag it instead.
- **Never change a URL** without a 301 in url-map.csv (section 9.1). Every internal link keeps its trailing slash.
- **Never build a URL by hand.** Use path() or href() for links and absolute() for canonical URLs (src/lib/urls.ts).
- **Never write a metadata export by hand.** Use buildMetadata() from src/lib/seo.ts.
- **Never remove** export const dynamic = 'error' from src/app/layout.tsx, and never turn on cacheComponents or output: 'export'.
- 'use client' only inside src/components/client/.
- Legacy images stay at /wp-content/uploads/.... Do not tidy those paths.
- Do not add an npm package without saying why in the commit or pull request.
- Do not run db:setup on a server, and never point your local site at the live database.
- Emails the site sends are written without hyphens or dashes (house style).

9. Common tasks

9.1 Add or change a redirect

```
# 1. edit data/url-map.csv: action=301, new_url=/the-new-page/ (trailing slash on both sides)
npm run redirects:build          # refuses chains, duplicates and loops
npm run dev                     # then open the old URL: it must 301 straight to the new one
```

After deploying: `curl -sI https://www.recycletechnologies.com/old-url/` must show 301 and the new location, in one hop.

9.2 Change the database structure

- Add a new numbered file: `db/013_what_it_does.sql` (next free number).
- Write it so it can run twice safely: `CREATE TABLE IF NOT EXISTS, ADD COLUMN IF NOT EXISTS, DROP CONSTRAINT IF EXISTS` before `ADD CONSTRAINT`.
- Locally: `npm run db:setup` (empty database) or `psql -U postgres -d rti_admin -f db/013_....sql` (restored copy).
- On dev and then live, run it with `psql` **before** the build (sections 10 and 11).

9.3 Add an image from Figma

Do not download Figma images by hand into the repo. Add the asset to `data/figma-assets.json` (node id and target path), then:

```
node scripts/fetch-figma-assets.mjs --missing
npm run verify:assets
```

9.4 Make page copy editable in Admin → Pages

Add the document to `src/content/registry.ts`, and in the component read it with `await content('key')` instead of importing the constant. Then check which fields actually show:

Windows PowerShell

```
$env:BASE="http://localhost:3000"; $env:ADMIN_USER="you@example.com"; $env:ADMIN_PASS="..."
node scripts/check-page-fields.mjs --write the-document-key
```

Run it against your local database only; it writes and discards drafts.

9.5 Admin accounts and roles

Role	Can do
Administrator	Everything, including People & access and site settings
Editor	Write, edit and publish any post; Admin → Pages; enquiries and lead workflow
Content writer	Write and edit their own posts; cannot publish
SEO	SEO fields on every post, Google & Tracking; not the body, not publishing
Sales agent	Only the enquiries assigned to them; logs phone enquiries
Ads manager	Subscribers, Enquiries, Lead workflow, Google & Tracking in full; content screens view only

Create or reset an account on any machine with `npm run user:add -- --login LOGIN --password "..."`
`--name "Name" --role ROLE` (run on the server from the dev or live folder). Normally an administrator does it in People & access.

10. Send changes to the dev server

Step 1, your computer: run the checks from section 6, then push.

```
git status
git add -A
git commit -m "Short description of the change"
git push origin main
```

Note. Agree with Asim how you push. The simple way is straight to main as above. The safer way for two people is a branch (`git checkout -b phase2/area-short-name`, push it, open a pull request on GitHub, merge it after review) and only then deploy main.

Step 2, the server:

```
ssh dev1@192.168.90.152
cd ~/apps/rti-website-dev
git pull origin main

# only when the change added a db/NNN file:
DB=$(grep -E '^DATABASE_URL=' .env.local | cut -d= -f2- | tr -d ' ')
psql -v ON_ERROR_STOP=1 "$DB" -f db/013_what_it_does.sql

npm ci && npm run build && npm run verify:static && pm2 restart rti-website-dev
sleep 8
curl -sI http://127.0.0.1:8106/ | head -1
```

Expect	
verify:static	"Build guard passed" and noindex flag: ON (staging)
curl	HTTP/1.1 200 OK
Browser	Check the change on https://dev.recycletechnologies.com

11. Send changes to the live site

Only deploy to live what has been checked on dev. Same server, different folder.

Step 1: back up the live database and note the current version.

```
ssh dev1@192.168.90.152
cd ~/apps/rti-website
DB=$(grep -E '^DATABASE_URL=' .env.local | cut -d= -f2- | tr -d ' ')
mkdir -p ~/backups/rti-live
pg_dump "$DB" | gzip > ~/backups/rti-live/before-deploy-$(date +%F-%H%M).sql.gz
git log -1 --oneline # write this id down: it is your rollback point
```

Step 2: new code, then any new migration.

```
git pull origin main
psql -v ON_ERROR_STOP=1 "$DB" -f db/013_what_it_does.sql      # only if there is one
```

Step 3: build and restart.

```
npm ci && npm run build && npm run verify:static && pm2 restart rti-website
sleep 10
pm2 ls
curl -sI http://127.0.0.1:8201/ | head -1
```

Expect

verify:static	"Build guard passed", noindex flag: off, deploy env: production . If it says noindex is on, stop: never ship noindex to live.
curl	HTTP/1.1 200 OK

Step 4: checks.

```
npm run verify:redirects -- --base http://127.0.0.1:8201      # ends with "All redirects pass"
curl -sI https://www.recycletechnologies.com/ | head -1
curl -s https://www.recycletechnologies.com/robots.txt | head -5 # must say Allow: /
```

Then open the pages you changed on <https://www.recycletechnologies.com> and look at /admin/.

If something is wrong: roll back

```
cd ~/apps/rti-website
git checkout ROLLBACK_ID_FROM_STEP_1
npm ci && npm run build && pm2 restart rti-website
```

Migrations are written to be harmless to the older code, so a rollback normally does not need the database restored. If it ever does:

```
gunzip -c ~/backups/rti-live/before-deploy-DATE.sql.gz | psql "$DB" # ask Asim first
```

When the problem is fixed, go back to the normal branch with `git checkout main`.

12. Facts about the server

- One VM, dev1@192.168.90.152, runs both dev and live. ~/apps is /home/sj/apps.
- There is no nginx on the VM and dev1 has no sudo. HTTPS and the domain names are handled by the admin team's proxy in front of it (dev to port 8106, live to port 8201).
- Both sites run under pm2. A crontab line (@reboot pm2 resurrect) brings them back after a reboot; after adding or renaming a pm2 process run `pm2 save`.
- Memory is tight (about 2.6 GB free). BUILD_CPUS in each .env.local limits build workers; do not raise it, and do not build dev and live at the same time.
- Each site has its own .env.local (database, session secret, SMTP, media folder, DEPLOY_ENV). Dev has NEXT_PUBLIC_NOINDEX=true and DEPLOY_ENV=staging; live has neither noindex nor staging. Do not print these files in shared places; they contain secrets.
- Useful: `pm2 ls`, `pm2 logs rti-website --lines 100`, `pm2 logs rti-website-dev --lines 100`, `free -h`, `df -h ~`.
- Live database backups go to ~/backups/rti-live/.

13. When something goes wrong

What you see	What it means / what to do
"The page copy tables are not in this database yet"	Migration 011 is missing. Run <code>psql ... -f db/011_page_content.sql</code> .
db:setup stops at 004 with <code>users_role_check</code>	You ran it on a restored copy. Do not; apply single migrations with <code>psql</code> (section 9.2).
Admin login works, then the next click says "Not signed in"	The site is on plain http at an IP address, so the secure cookie is not sent back. Use https or localhost.
"PostgreSQL refused that password"	The password in <code>.env.local</code> is not the one set when PostgreSQL was installed.
Signed out after every restart	<code>ADMIN_SESSION_SECRET</code> keeps changing. Set it once in <code>.env.local</code> .
Build fails talking about noindex	A production build found <code>NEXT_PUBLIC_NOINDEX=true</code> . On dev, <code>DEPLOY_ENV=staging</code> must also be set; on live, noindex must be absent.
Build is very slow or pages time out on the server	Out of memory. Make sure only one build runs; keep <code>BUILD_CPUS</code> as set.
Blog is empty locally	Your database is empty (option A). Use option B in section 5.3.
Images missing locally	<code>var/uploads/</code> is empty. Copy it from dev (section 5.3), or run <code>node scripts/fetch-figma-assets.mjs --missing</code> for design images.
An enquiry email did not arrive	SMTP settings in that site's <code>.env.local</code> . Admin → Enquiries → the enquiry's Activity shows assignment email results.
<code>git pull</code> on the server refuses because of local changes	Someone edited files on the server. Do not force it; ask Asim before discarding anything.

14. Checklist for your first day

- ☐ GitHub access works: `git clone` succeeded.
- ☐ Node 22.22+, Git and PostgreSQL installed.
- ☐ `.env.local` created with `DATABASE_URL` and `ADMIN_SESSION_SECRET`.
- ☐ Database loaded (option B) and your own admin login created.
- ☐ `npm run dev` shows the site at `localhost:3000` and you can sign in at `/admin/`.
- ☐ `npm run build` and `npm run verify:static` pass on your machine.
- ☐ Read `CLAUDE.md`.
- ☐ SSH login to `192.168.90.152` works (for deploying).
- ☐ Agreed with Asim: push straight to main, or branches and pull requests.